

PDF-отчёт о защите капстоуна

Дисциплина «Системное администрирование»

Студент	Дворяшин Анатолий Васильевич
Репозиторий	https://github.com/Exzhnik/my-sysadmin-scripts
Выбранная дорожка	Продвинутый
Среда проверки	Удалённая Ubuntu-машина sysadmin-host
Дата свежего прогона	26 сентября 2026 г.

Цель отчёта — подтвердить свежим end-to-end прогоном, что все компоненты проекта работают совместно: скрипт, контейнер, хранилище, reverse proxy, TLS, systemd и журналы.

1. Технический чеклист

Все команды выполнены на машине sysadmin-host 26.09.2026. Вывод сокращён только там, где повторяющиеся строки не влияют на доказательство.

1. Git-репозиторий, история и README

Команда

```
git log --oneline -10
git remote -v
```

Фактический вывод

```
22f8e9c fix: учтены штатные коды остановки контейнера
55a452e docs: дополнено описание развёртывания
a7c443e feat: добавлена systemd-служба контейнера
221d108 feat: настроен Nginx с HTTPS
e85f50b feat: добавлен HTTP-сервис на порту 8080
fc0d57d feat: добавлен запуск через Docker Compose с томом
5c0727e feat: добавлена контейнеризация скрипта
87c4ca9 docs: добавлен README и пример вывода
6414baf fix: добавлена проверка записи в лог
84cf9cb feat: добавлен сбор ресурсов каждые 5 секунд
origin git@github.com:Exzhnik/my-sysadmin-scripts.git (fetch)
```

История содержит более трёх осмысленных коммитов и показывает развитие решения. Репозиторий связан с GitHub; README находится в корне проекта.

2. Скрипт автоматизации

Команда

```
TMPDIR_CAP=$(mktemp -d)
cd "$TMPDIR_CAP"
timeout 7 /home/student/my-sysadmin-scripts/script.sh
tail -n 20 monitor.log
```

Фактический вывод

```
--- 2026-09-26 16:12:50 ---
Mem: 1.9Gi total, 262Mi used, 656Mi free
/dev/sda1  9.6G  3.7G  5.9G  39% /
/dev/md0   463M   24K  427M   1% /mnt/raid
/dev/mapper/vg_data-lv_logs 172M 24K 158M 1% /mnt/logs
16:12:50 up 18:01, load average: 0.00, 0.00, 0.00
```

Скрипт самостоятельно собирает дату, память, файловые системы и uptime и записывает результаты в monitor.log. Проверка выполнена вне контейнера во временном каталоге.

3. Dockerfile, образ и контейнер

Команда

```
docker build -t my-script .
docker ps -a --filter name=my-app
docker inspect -f 'image={{.Image}} status={{.State.Status}}' my-app
docker image inspect -f '{{.Id}}' my-script:latest
```

Фактический вывод

```
Successfully built f949eaaeb17
Successfully tagged my-script:latest
my-app my-script:latest Up 17 seconds 0.0.0.0:8080->8080/tcp
image=sha256:f949eaaeb17... status=running
sha256:f949eaaeb17...
```

Dockerfile успешно собирается в образ my-script:latest. Контейнер my-app запущен из этого же свежего образа: идентификаторы совпадают; порт 8080 опубликован.

4. RAID 1 и LVM

Команда

```
cat /proc/mdstat
sudo lvs
sudo vgs
df -h | grep -E "raid|logs"
```

Фактический вывод

```
md0 : active raid1 loop7[1] loop6[0]
      523264 blocks super 1.2 [2/2] [UU]
lv_logs vg_data -wi-ao---- 200.00m
vg_data 1 PV 1 LV VSize 508.00m VFree 308.00m
/dev/md0          463M 24K 427M 1% /mnt/raid
/dev/mapper/vg_data-lv_logs 172M 24K 158M 1% /mnt/logs
```

Массив RAID 1 активен и исправен: оба участника отмечены UU. Группа vg_data и том lv_logs активны; обе файловые системы смонтированы.

5. Nginx как обратный прокси

Команда

```
sudo nginx -t
```

Фактический вывод

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Действующая конфигурация Nginx синтаксически корректна. Запросы из HTTPS location перенаправляются на приложение по адресу 127.0.0.1:8080.

6. HTTPS и TLS-сертификат

Команда

```
curl -kI --max-time 10 https://127.0.0.1/monitor.log
```

Фактический вывод

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
```

Ответ HTTP/1.1 200 OK подтверждает, что TLS подключён, Nginx принимает HTTPS-запрос и успешно передаёт его контейнеру.

7. systemd-служба

Команда

```
sudo systemctl is-enabled my-app
sudo systemctl status my-app --no-pager
```

Фактический вывод

```
enabled
• my-app.service - My monitoring container
Loaded: loaded (/etc/systemd/system/my-app.service; enabled)
Active: active (running) since Sat 2026-09-26 16:34:05 MSK
Main PID: 69858 (docker)
```

Служба включена в автозапуск и находится в состоянии active (running). systemd управляет контейнером my-app через Docker.

8. Журналы приложения и Nginx

Команда

```
curl -ksS -o /dev/null https://127.0.0.1/monitor.log
sudo journalctl -u my-app -n 20 --no-pager
sudo tail -n 20 /var/log/nginx/access.log
```

Фактический вывод

```
Sep 26 16:34:22 sysadmin-host docker[69858]: "HEAD /monitor.log HTTP/1.0" 200
Sep 26 16:34:22 sysadmin-host docker[69858]: "GET /monitor.log HTTP/1.0" 200
127.0.0.1 - - [26/Sep/2026:16:34:22 +0300] "HEAD /monitor.log HTTP/1.1" 200
127.0.0.1 - - [26/Sep/2026:16:34:22 +0300] "GET /monitor.log HTTP/1.1" 200
```

Свежий тестовый запрос с кодом 200 виден и в журнале systemd, и в access.log Nginx. Это подтверждает прохождение запроса через всю цепочку.

2. Архитектурные решения

2.1. Почему выбран вариант скрипта

Выбран вариант Б — «Мониторинг ресурсов»: раз в пять секунд скрипт фиксирует дату, использование памяти и дисков, а также uptime. Такой вариант хорошо подходит для демонстрации автоматизации: результат накапливается во времени, легко проверяется обычными средствами Linux и естественно публикуется через простой HTTP-сервис. В скрипт добавлена явная проверка возможности записи, чтобы ошибка прав доступа не оставалась незамеченной.

2.2. RAID 1 и LVM

RAID 1 собран из двух loop-устройств и создаёт отказоустойчивый слой за счёт дублирования данных. Цена этого решения — полезная ёмкость равна объёму одного участника массива. LVM собран отдельно на третьем loop-устройстве: группа `vg_data` содержит логический том `lv_logs`. LVM решает другую задачу — гибкое выделение и последующее изменение объёма, но сам по себе не дублирует данные. Поэтому RAID 1 и LVM здесь не конкурируют: первый демонстрирует устойчивость, второй — управление пространством.

2.3. Способ получения сертификата

Для локальной учебной среды выбран самоподписанный сертификат OpenSSL. Он позволяет полностью проверить TLS без доменного имени и внешнего центра сертификации. Клиент предупреждает о недоверенном издателе, поэтому при тесте используется `curl -k`. Для публичного сервиса предпочтительнее сертификат Let's Encrypt с автоматическим продлением, но для него нужны доступный домен и подтверждение владения.

2.4. Рассмотренные альтернативы

Запуск контейнера только вручную через Docker CLI был отклонён, потому что он не обеспечивает автозапуск после перезагрузки; выбран `systemd`. Прямая публикация порта 8080 не решает задачи TLS и централизованного журналирования, поэтому перед приложением установлен Nginx. Docker Compose сохранён как удобный способ локального запуска, однако системная служба управляет финальным контейнером напрямую и делает состояние видимым стандартными командами `systemd`.

3. Что не получилось с первого раза

После перезагрузки машины файлы-образы в `/mnt/raid-lab` сохранились, но привязки к loop-устройствам исчезли. Поэтому `/dev/md0` и группа LVM не отображались, а точки `/mnt/raid` и `/mnt/logs` были не смонтированы. Данные не пересоздавались: образы повторно подключены через `losetup --show`, RAID собран командой `mdadm --assemble`, группа LVM активирована через `vgchange -ay`, после чего оба тома смонтированы. Итог проверен не только отсутствием ошибок, но и выводом `mdstat`, `lvs` и `df -h`.

Ранее штатная остановка Docker-контейнера могла давать коды 137/143 и восприниматься `systemd` как сбой. В `unit`-файл добавлен параметр `SuccessExitStatus=0 137 143`; история Git сохраняет отдельный исправляющий коммит, что показывает реальный ход работы.

4. Итоговая самопроверка

Пункт	Результат
Публичный репозиторий и история	Подтверждено
Скрипт автоматизации	Подтверждено
Dockerfile, образ и контейнер	Подтверждено
RAID 1 и LVM	Подтверждено
Nginx reverse proxy	Подтверждено
TLS и HTTPS 200	Подтверждено
systemd enabled/active	Подтверждено
journalctl и Nginx access.log	Подтверждено

Свежий end-to-end прогон подтвердил работоспособность всех восьми пунктов технического чеклиста.